

ASSIGNMENT 3

Разработка и интеграция научного модуля Smart Waste Astana

Software Development and Integration

Тема исследования: Разработка цифровой платформы для автоматизации сбора и вывоза мусора в городе Астана

Магистрант: Хаймульдин Нурдаулет Газизұлы

Группа: CSE-2504M

Астана, 2026

Цель и результат работы

Разработан воспроизводимый программный модуль для обработки телеметрии мусорных контейнеров, прогноза заполненности и моделирования вывоза. Практическая часть показывает полный цикл инженерной работы: требования, выбор технологий, Git, публичную публикацию, автоматические проверки и доставку проверенного архива.

В отчёте рассмотрены организация разработки, критерии выбора инструментов, системы контроля версий, CI/CD и особенности научного программного обеспечения. Решения объясняются на примере реализованного модуля, который поддерживает два и пять алгоритмических агентов и работает без GPU.

Демонстрация использует синтетические данные. Результаты характеризуют поведение программы в заданном эксперименте и не доказывают эффективность муниципального вывоза в реальной Астане.

1 Организация процесса разработки

Разработка начинается с постановки исследовательского вопроса: можно ли на основе истории заполненности определить контейнеры, которые следует вывезти на заданном горизонте, и построить допустимые рейсы? Для Assignment 3 этот вопрос преобразован в небольшой модуль с измеримыми входами и выходами. Вместо всей городской платформы реализована вычислительная часть, которую можно независимо запускать и тестировать.

На этапе требований определены формат CSV, единицы измерения, минимальное число наблюдений, ограничения входных значений и ожидаемые файлы результата. Критерий готовности: одинаковый набор данных и параметры дают одинаковый результат; выбранные контейнеры обслуживаются ровно один раз; объем каждого рейса не превышает вместимости машины; прогноз оценивается на данных, исключённых из обучения.

При проектировании выделены модели, чтение данных, агенты, экспорт и командный интерфейс. Такой подход уменьшает связанность: формат представления можно изменить без переписывания вычислений. Далее реализованы базовые агенты анализа и приоритетов, затем прогноз, маршрутизация и оценка. Каждое расширение связано с конкретной функцией и проверяемым поведением.

После реализации выполнены модульные и интеграционные проверки, сборка автономного ZIP и запуск из распакованной копии. Публикация оформлена через ветку и pull request. Автоматизация повторяет основные локальные проверки на разных операционных системах. Поддержка организована через Issues с критериями приёмки; изменение алгоритма должно сопровождаться проверкой результата и обновлением документации.

Для небольшого исследовательского проекта подходит итерационная организация: рабочая версия создаётся рано, а следующие итерации расширяют её возможности. Этапы не считаются однократными: новая телеметрия может изменить требования к очистке данных, а результаты тестирования — архитектурное решение. Завершённая итерация должна давать код, документацию и воспроизводимый эксперимент.

2 Критерии выбора технологий

Инструменты выбирались по пяти критериям: возможность запуска на обычном ноутбуке, прозрачность вычислений, воспроизводимость, простота автоматического тестирования и достаточность для текущего исследовательского масштаба. Отдельно учитывалась работа на Windows, поскольку сложная установка окружения затрудняет демонстрацию и проверку задания.

Python выбран для компактного исследовательского модуля. Используются стандартные csv, json, math, random, dataclasses и unittest. Это позволяет выполнять демонстрацию сразу после установки Python 3.11 или новее. Внешние runtime-библиотеки отсутствуют. Для стандартной установки пакета предусмотрен pyproject.toml; запуск из исходников и ZIP не требует сборочного backend [1, 2].

Отказ от тяжёлых библиотек соответствует масштабу задачи: линейный тренд и расстояние Haversine вычисляются небольшими явными функциями. Если объём телеметрии увеличится, можно добавить NumPy и pandas для обработки массивов или scikit-learn для более сложного прогноза. Подключение библиотеки должно решать измеренную проблему, а не только увеличивать технологический стек.

HTML и CSS используются для отчёта с таблицами и графиками. Файл открывается офлайн и не требует веб-сервера. Git фиксирует версии исходников, GitHub предоставляет публичную публикацию и совместную работу, GitHub Actions выполняет проверки. Выбор этих инструментов позволяет показать единый процесс разработки и доставки [3, 4].

Инструмент	Польза в проекте	Практическое ограничение
Python и unittest	Прозрачные алгоритмы и тесты	Нет ускорения больших массивов
HTML и CSV	Просмотр и экспорт результатов	Статический снимок эксперимента
Git и GitHub	История, PR, Issues и публикация	Нужна дисциплина версий
GitHub Actions	Windows и Linux, доставка ZIP	Зависимость от внешнего сервиса

3 Контроль версий и размещение проекта

Система контроля версий сохраняет изменения файлов и позволяет восстановить состояние проекта для конкретного эксперимента. Git использует распределённую модель: рабочая копия содержит историю и поддерживает локальные коммиты. Это полезно при исследовательской работе без постоянного подключения к сети [3].

Git и GitHub выполняют разные функции. Git отвечает за коммиты, ветки и объединение изменений. GitHub размещает репозитории и добавляет pull requests, Issues, проверку изменений и автоматизацию. Репозиторий может использовать Git без GitHub, а выбор платформы размещения не меняет базовые алгоритмы контроля версий.

В качестве альтернатив Git можно рассматривать Mercurial и Subversion. Mercurial тоже использует распределённую историю; Subversion основан на центральном репозитории. Для задания выбран Git, поскольку он уже используется в родительском проекте и непосредственно поддерживается GitHub Actions. Сравнивать следует модель совместной работы и доступные инструменты, а не считать платформу размещения отдельной VCS.

Модуль опубликован в публичном репозитории NuKANANA/waste, в папке research/smart-waste-astana. Такой способ сохраняет связь с существующей платформой и позволяет запускать исследовательскую часть независимо от Go-сервисов. Разработка оформлена отдельной веткой assignment-3/smart-waste-astana и pull request №1. История содержит отдельные коммиты с кодом, тестами, документацией и автоматизацией.

README объясняет запуск, данные и ограничения. MIT распространяется на новый модуль и его синтетические данные; условия других частей репозитория не изменяются. Issues №2 и №3 описывают следующие исследовательские этапы: валидацию на реальных данных и маршрутизацию по дорожной сети. Задачи имеют критерии приёмки, поэтому их можно проверить при завершении.

Объект	Назначение
Коммит	Воспроизводимая версия кода
Ветка	Изолированная разработка изменения
Pull request	Обзор изменений и результаты CI
Issue	Проблема или следующий этап работы

4 Концепции непрерывной интеграции и доставки

Непрерывная интеграция, CI, означает частое объединение небольших изменений с автоматической проверкой. Проверки должны выявлять ошибки до того, как новая версия станет основой следующих экспериментов. Для научного модуля недостаточно проверить синтаксис: нужно убедиться, что прогноз не использует контрольную дату, а маршрут соблюдает физическое ограничение объёма.

Непрерывная доставка, continuous delivery, формирует готовый проверенный артефакт, который можно передать пользователю. Continuous deployment означает автоматическое развёртывание изменения в рабочей среде. В данном проекте конечный продукт — автономный ZIP исследовательского инструмента; веб-сервис не развёртывается. Доставка через Actions artifact и отдельный release workflow соответствует этому типу продукта.

Workflow состоит из события, jobs и steps. Событие push или pull request запускает проверки после изменения модуля. Матрица запускает одну и ту же последовательность на Ubuntu с Python 3.11 и 3.13 и Windows с Python 3.12. Это обнаруживает различия кодировок, путей и платформ, которые один локальный запуск может не выявить [4].

Проверки включают компиляцию исходников, unittest, демонстрационный эксперимент, сборку ZIP и запуск из распакованного архива. Job доставки имеет зависимость needs: test, поэтому проверенный архив публикуется только после успешного завершения всей матрицы. Выходные результаты эксперимента сохраняются вместе с архивом [5].

Обычный CI использует contents: read. Отдельный release workflow запускается при добавлении VERSION в main или вручную и получает contents: write только для публикации релиза. Перед доставкой он повторяет тесты. Версия фиксирована как smart-waste-v1.0.0; повторный запуск сохраняет существующий релиз. Пароли и пользовательские токены в исходниках не хранятся.

5 Особенности научного программного обеспечения

Научное программное обеспечение должно позволять проверить путь от данных к выводу. Поэтому в отчёте JSON сохраняются параметры сценария, версия программы, контрольная сумма входного CSV и порядок выполненных агентов. Это позволяет установить, какой набор данных и алгоритм дали конкретный результат, и сравнить повторные запуски.

Воспроизводимость обеспечивается фиксированным seed, датами и детерминированным разрешением равных расстояний по идентификатору контейнера. Экспорт не добавляет текущее время, поэтому одинаковые входы дают одинаковые файлы побайтно. Сборка архива использует фиксированную дату записей и явный список включаемых каталогов. Рабочие окружения, Git-метаданные и временные файлы не входят в дистрибутив.

Качество данных проверяется до вычислений. Отклоняются пустые значения, дубликаты контейнер–дата, NaN, Infinity, неверные координаты, отрицательная ёмкость и заполненность вне диапазона 0–100%. Для одного контейнера метаданные должны оставаться постоянными. Все контейнеры имеют общую последнюю дату, чтобы планирование сравнивало одно состояние времени.

Падение заполненности обычно означает вывоз, сброс датчика или ошибку. Модель этого прототипа предполагает один цикл накопления, поэтому такой ряд отклоняется с объяснением. Для реальной телеметрии необходим отдельный этап сегментации циклов и документированная обработка пропусков. Скрытая очистка данных могла бы изменить выводы и затруднить их воспроизведение.

Важна граница между демонстрацией программы и научной валидацией. Синтетический набор удобен для проверки известных сценариев, но его распределение задано генератором. Низкая ошибка на такой выборке не доказывает переносимость на реальный город. Перед прикладными выводами нужны разрешённые реальные измерения, несколько временных периодов, baseline и анализ неопределённости.

6 Архитектура многоагентного модуля

Модуль реализует последовательную многоагентную архитектуру с общим типизированным контекстом. Агент — самостоятельный компонент принятия решения, имеющий имя и метод `run(context)`. В этом проекте решения основаны на явных правилах, регрессии и эвристике; агенты не являются распределёнными LLM-сервисами и не обучают нейронную сеть.

Context содержит исходные наблюдения, конфигурацию, состояния контейнеров, рейсы, метрики и trace. AnalysisAgent создаёт состояния; ForecastAgent изменяет прогноз; PriorityAgent выбирает контейнеры; RouteAgent строит рейсы; EvaluationAgent оценивает прогноз. Оркестратор последовательно вызывает компоненты и сохраняет их имена.

Режим двух агентов выполняет анализ и определение приоритетов по текущему заполнению. Он полезен как минимальная работающая версия и для сравнения с прогнозным планированием. Режим пяти агентов добавляет прогноз, рейсы и оценку. Параметр `--agents` принимает 2 или 5, а программный интерфейс допускает произвольный список реализаций Agent.

Расширение не требует изменения загрузчика CSV или экспортера. Например, агент предупреждений может добавить число контейнеров с прогнозом выше 95% в metrics. При этом порядок пользовательских компонентов должен учитывать зависимости: планирование рейсов возможно только после анализа и отбора. Типизированный контракт облегчает понимание, но сам по себе не проверяет корректность пользовательского агента.

Компонент	Вход и выход решения
AnalysisAgent	История → текущие состояния и рост
ForecastAgent	Рост и горизонт → прогноз
PriorityAgent	Прогноз и порог → отбор
RouteAgent	Отбор и ёмкость → рейсы
EvaluationAgent	История → holdout MAE и baseline

7 Алгоритмы прогнозирования и планирования

Для контейнера известны даты t_i и проценты заполненности y_i . AnalysisAgent оценивает скорость роста методом наименьших квадратов: $b = \Sigma(t_i - \bar{t})(y_i - \bar{y}) / \Sigma(t_i - \bar{t})^2$. Время выражено в днях от первой даты, поэтому нерегулярные интервалы не превращаются в одинаковые шаги. При неизменном ряде скорость равна нулю; отрицательная скорость ограничивается нулём в рамках модели одного цикла.

ForecastAgent строит прогноз $\hat{y} = \min(100, y_{\text{last}} + b \times h)$, где h — горизонт в днях. Это прогноз тренда, привязанный к последнему измерению. Например, для 50%, 60%, 70% с интервалом один день скорость равна 10 процентным пунктам в день; прогноз через два дня равен 90%. Ограничение 100% сохраняет физический смысл объёма.

PriorityAgent выбирает контейнер, если прогноз не ниже порога, по умолчанию 80%. Дополнительно назначается категория critical при прогнозе от 95%, high для выбранных контейнеров, medium от 60% среди невыбранных и low для остальных. При изменении порога решение о вывозе определяется полем selected, а не только текстовым названием категории.

Ожидаемый объём вычисляется как ёмкость контейнера, умноженная на прогнозную долю заполненности. RouteAgent начинает рейс в условном депо и выбирает ближайший контейнер, который помещается в оставшуюся вместимость машины. Когда допустимых контейнеров нет, рейс возвращается в депо и начинается новый. Каждый выбранный контейнер посещается ровно один раз.

Расстояние Haversine описывает геометрическую дистанцию на сфере с радиусом 6371,0088 км. Учитываются путь от депо и возврат. Эвристика ближайшего соседа не гарантирует глобальный минимум длины; модель не учитывает дороги, пробки, время обслуживания и длину смены. Если один контейнер превышает вместимость машины, расчёт отклоняется вместо выдачи недопустимого рейса.

8 Данные и воспроизводимый эксперимент

Демонстрационный CSV создан генератором с `seed=42`: 24 контейнера, по 7 наблюдений за 1–7 октября 2026 года, всего 168 строк. Координаты выбраны в прямоугольнике $51,10$ – $51,18^\circ$ северной широты и $71,37$ – $71,49^\circ$ восточной долготы. Они являются вымышленными точками в условной географии Астаны. Названия районов служат подписями, а не результатом пространственного определения границ.

Для каждого контейнера случайно задаются ёмкость, начальное заполнение и скорость накопления. К ежедневному приросту добавляется небольшое возмущение. Используются ёмкости 1,1, 2,5 и 4,0 м³; сброс после вывоза не моделируется. Генератор, параметры и происхождение набора описаны в `data_provenance.md`. Реальные муниципальные измерения в работу не включены.

Эксперимент запускается командой `python main.py demo`. По умолчанию выполняются пять агентов, горизонт равен одному дню, порог — 80%, вместимость машины — 12 м³. Дополнительный запуск с `--agents 2` показывает отбор только по текущему состоянию. Изменение `--horizon` или `--truck-capacity` позволяет исследовать зависимость решения от параметров.

Прогноз оценивается методом `last-date holdout`. Для каждого контейнера последняя дата исключается, а скорость оценивается только по предыдущим датам. Прогноз сравнивается с последним фактическим значением. `Baseline persistence` повторяет последнее значение обучающей части без учёта роста. MAE равна среднему абсолютному отклонению в процентных пунктах.

Для планирования будущего вывоза используется вся имеющаяся история. Это отдельная задача от оценки на `holdout`: нельзя подставлять прогноз для будущего дня в качестве результата тестирования. JSON сохраняет индивидуальные контрольные ошибки, что позволяет проверить агрегированную метрику. В данном эксперименте используется одна контрольная дата на контейнер, поэтому следующий этап — оценка на нескольких периодах.

9 Результаты проверки и автоматизация

Локальные проверки завершились успешно: выполнено 30 поведенческих тестов, демонстрационный запуск и проверка автономного ZIP. Набор тестов проверяет известный линейный ряд, нерегулярные даты, ограничение прогноза, включение граничного порога, отсутствие утечки контрольного наблюдения, вместимость рейсов, возврат в депо и обслуживание контейнеров ровно один раз.

Проверки данных охватывают дубликаты, изменение метаданных, сброс заполненности, неверные числа, даты и заголовки. Интеграционные тесты вызывают CLI, читают файлы результатов и проверяют побайтную воспроизводимость экспорта. HTML экранирует пользовательские подписи; CSV предотвращает интерпретацию подписей как формул электронных таблиц.

9 октября 2026 года workflow Smart Waste CI and Delivery успешно завершился в GitHub. Проверки выполнены на Ubuntu с Python 3.11 и 3.13 и на Windows с Python 3.12. Job доставки сформировал доступный артефакт после успешной матрицы. Подтверждённый запуск: github.com/NuKANANA/waste/actions/runs/37919752690.

Результаты ниже вычислены программой для seed=42 и стандартных параметров. Они подтверждают работоспособность демонстрации. Сравнение с persistence характеризует только синтетическую контрольную выборку и не является оценкой городского экономического эффекта.

Показатель	Результат демонстрации
Контейнеры и наблюдения	24 контейнера, 168 наблюдений
Выбрано к вывозу	8
Рейсы	2
Объём	18.097 м ³
Расстояние по прямой	35.814 км
MAE прогноза	0.646 п.п.
MAE persistence	7.139 п.п.

10 Выводы и самоанализ

В работе реализован законченный небольшой научный инструмент, связанный с темой автоматизации сбора и вывоза мусора. Модуль обрабатывает CSV, моделирует рост заполненности, объясняет отбор контейнеров, формирует допустимые по объёму рейсы и оценивает прогноз на исключённой дате. Минимальная версия содержит два агента, расширенная — пять; интерфейс допускает дальнейшее расширение.

Инженерная часть включает публичную публикацию, осмысленную историю коммитов, README, лицензию, Issues, pull request, автоматические проверки и доставку ZIP. Выбор Python со стандартной библиотекой сделал демонстрацию независимой от GPU, облачных API и длительной установки библиотек. Архив можно распаковать и запустить на обычном компьютере с Python 3.11 или новее.

Основные трудности связаны с воспроизводимостью окружения, корректностью временной оценки и физическими ограничениями модели. Различие путей и кодировок проверяется матрицей Windows/Linux. Утечка будущего наблюдения исключается временным holdout и отдельным тестом с контрольным выбросом. Недопустимый объём рейса предотвращается явным ограничением вместимости и проверкой слишком большого контейнера.

Текущие ограничения определяют границы выводов. Синтетическая телеметрия не отражает реальную изменчивость города; линейный прогноз не учитывает сезонность; расстояние по прямой не соответствует маршруту по улицам; число рейсов не ограничено временем смены. Эти условия не скрываются в метриках и документации. Поэтому пока нельзя утверждать снижение расходов, пробега или числа переполнений в реальной эксплуатации.

Следующие этапы представлены в Issues: получить разрешённые реальные измерения, описать качество датчиков, сегментировать циклы вывоза, оценить прогноз на нескольких временных периодах и добавить дорожный граф. Многоагентная структура позволяет включить такие возможности отдельными компонентами. Работа показывает, как организация разработки, тестирование и CI/CD делают исследовательский результат проверяемым и пригодным для дальнейшего развития.

11 Источники и ссылки на материалы

[1] Python Software Foundation. The Python Standard Library. <https://docs.python.org/3/library/> — справочник стандартных модулей Python. Дата обращения: 09.10.2026.

[2] Python Packaging Authority. Writing your pyproject.toml. <https://packaging.python.org/en/latest/guides/writing-pyproject-toml/> — описание конфигурации Python-проекта. Дата обращения: 09.10.2026.

[3] Chacon S., Straub B. Pro Git. Getting Started — About Version Control. <https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control> — модель контроля версий. Дата обращения: 09.10.2026.

[4] GitHub Docs. Building and testing Python. <https://docs.github.com/en/actions/tutorials/build-and-test-code/python> — конфигурация CI для Python. Дата обращения: 09.10.2026.

[5] GitHub Docs. Store and share data with workflow artifacts. <https://docs.github.com/en/actions/tutorials/store-and-share-data> — доставка артефактов. Дата обращения: 09.10.2026.

Практическая часть:

<https://github.com/NuKANAHA/waste/tree/main/research/smart-waste-astana>

Конфигурация CI/CD:

<https://github.com/NuKANAHA/waste/blob/main/.github/workflows/smart-waste-ci.yml>

Конфигурация релиза:

<https://github.com/NuKANAHA/waste/blob/main/.github/workflows/smart-waste-release.yml>

Pull request: <https://github.com/NuKANAHA/waste/pull/1>

Подтверждённый успешный CI:

<https://github.com/NuKANAHA/waste/actions/runs/37919752690>

Задачи: <https://github.com/NuKANAHA/waste/issues/2> и

<https://github.com/NuKANAHA/waste/issues/3>